



Ele tem uma equipe *quando precisa*.

Em *12min* você sabe se PRECISA de mais de 1 agente — e como criar um time digital quando precisar. Maioria absoluta dos casos: *1 agente bem configurado resolve*. Multi-agente é refinamento, não default. E o custo verdadeiro não é o setup — é a *manutenção 5x*.

A-HA DESTA AULA

A-HA #9 ENTREGUE

"Ele tem uma equipe quando precisa — e eu sei o caminho honesto pra montar."

COMO BRUNO CHEGOU EM MULTI-AGENTE · TIMELINE REAL

De 1 agente pra *3 em 6 meses*.

Pra entender quando faz sentido ter mais de 1 agente, conta a história de como Bruno chegou nisso na Pixel. Não foi planejado — foi necessidade.

JANEIRO

Só Amora

Amora é Chief of Staff — segundo cérebro do Bruno. Roda Telegram pessoal, Slack interno da Pixel, calendar, Notion, GitHub. Resolve 90% da operação dele. **1 agente, 1 identidade, 1 workspace.** Custo: ~R\$ 200/mês.

MARÇO

Nasce openclawzinho

Comunidade Telegram do curso OpenClaw passa de 100 → 1.000 alunos em 2 meses. Amora começa a confundir contexto pessoal do Bruno com suporte de comunidade. **Decisão:** agente novo dedicado a atender comunidade. openclawzinho nasce com SOUL próprio (alegre, casual, paciente), workspace separado, foco em FAQ + escalar pra humano. **2 agentes, papéis distintos.**

ABRIL

Léo entra como gestor de equipe

Time da Pixel cresce pra 8 pessoas. Bruno percebe que precisa de agente **compartilhado** entre os sócios — calendário comum, decisões registradas, Notion da empresa. Léo nasce como agente de equipe — todo sócio fala com ele, ele agrega. Diferente de Amora (pessoal do Bruno) e openclawzinho (público da comunidade). **3 agentes, 3 escopos: pessoal · público · equipe.**



Custo total dos 3: ~R\$ 500/mês. **Substituem:** assistente virtual da Pixel (~R\$ 4k/mês), tempo do Bruno respondendo comunidade (~10h/sem), e coordenação interna manual entre sócios (incalculável).



Lição da timeline: não comece com 3 agentes. Comece com 1, deixe ele saturar (sinais: confusão de contexto, escopo diluído, performance caindo), aí divida. A13 ensina **quando dividir** e **como dividir bem** — não **se** dividir.

PRE-FLIGHT CHECK

Antes de *apertar play*.

A13 amarra com 3 aulas anteriores. Releitura rápida ajuda — mas não é pré-requisito duro. Tem em mãos:

- ✓ **A5 fresca** — você já criou `AGENTS.md` no day 1 com framing "preparado pra equipe". Hoje ele ganha substância
- ✓ **A10 absorvida** — princípios de segurança operacional aparecem aqui (escopo por agente, custo de credenciais, monitoramento)
- ✓ **A4 lembrada** — `requireMention: true` resolve a pergunta clássica "se 2 bots no mesmo grupo, eles se respondem?"
- ✓ **Telegram aberto** em paralelo · você vai ver demo de subagente rodando
- ✓ **Cheatsheet do kit** aberto em outra aba — [multi-agente.md](#) tem 6 prompts prontos pra colar (incluindo "criar agente paralelo dedicado a 1 canal" e "migrar contexto saturado")

⚠ **Aluno PME atenção:** A13 NÃO é "vá criar 5 agentes". É exatamente o contrário — entender quando vale, e ter humildade técnica de ficar com 1 quando 1 resolve. *Estética > função* é a forma mais cara de overengineering.

ESTRUTURA DA AULA EM VÍDEO

O que você vai aprender em *7 blocos*.

🕒 1.5 MIN

Bloco 1 — Pergunta + custo real de manutenção

1 agente ou vários? Resposta honesta: maioria PME, 1 resolve. **Multiplicador 5x** de manutenção (SOULs · credenciais · monitoramento · skills · comunicação). 4 sinais de que vale evoluir mesmo assim.

🕒 2 MIN

Bloco 2 — Subagente vs Agente Paralelo

Distinção fundamental + tabela de prós/contras. **Subagente** = background run com identidade limitada (só herda `AGENTS.md` + `T00LS.md`). **Agente paralelo** = entidade separada com SOUL/IDENTITY/USER próprios.

🕒 2 MIN

Bloco 3 — Subagente em ação · pesquisador

Demo prático: `/subagents spawn researcher "..."`. Aurora dispara, pesquisador roda async em sandbox isolado, retorna síntese. **Tela split** — Aurora limpa em cima, subagente cheio de fontes embaixo.

🕒 2.5 MIN

Bloco 4 — Arquitetura + 4 caminhos de comunicação

Onde agentes vivem (Managed = 1 OpenClaw, N agentes mesmo workspace). 4 caminhos REAIS de comunicação (subagente spawn · mention · arquivo compartilhado · cron). **NÃO existe DM interna direta entre agentes** em runtime.

🕒 1.5 MIN

Bloco 5 — AGENTS.md como organograma

Aquele `AGENTS.md` da A5 ganha substância. Quem é cada agente · quem chama quem · onde cada um vive · permissões cruzadas · governance. **Source of truth vivo** do time digital.

🕒 1.5 MIN

Bloco 6 — Caso real Pixel + custo

3 agentes ativos hoje (Amora COS · openclawzinho · Leonardo Da Vinci) + subagentes on-demand. Custo controlado por design: caps mensais, audit log, digest semanal. **Custo previsível, sem surpresa.**

🕒 1 MIN

Bloco 7 — Quando NÃO + recap + amarração A14

6 sintomas de overengineering. Recap em 4 linhas. **Amarração A14:** "se você tem 1 OU vários agentes operando 24/7 — como você OBSERVA tudo?" → Mission Control.

APROFUNDAMENTO DO BLOCO 1

Custo real de manutenção — *multiplicador 5x.*

Aluno PME chega na A13 quase sempre pensando "vou criar 3 agentes amanhã". A primeira coisa que A13 entrega é honestidade: **3 agentes ≠ 3x trabalho. Mais perto de 5x.** Não pelo setup — pela manutenção contínua.

O que multiplica quando você tem 3 agentes

EIXO	COM 1 AGENTE	COM 3 AGENTES
Identidade		
Credenciais		
Monitoramento		
Skills		
Comunicação entre eles		



Princípio invariante: multiplicar agentes multiplica manutenção. Comece com 1. Evolua quando provar que precisa — e que aceita o custo. *Estética > função* é a forma mais cara de overengineering em IA.

4 sinais de que VALE evoluir mesmo assim

1

Agente único tá lento

Context rot · threads pesadas · respostas demoram porque ele tem que carregar tudo. Sinal técnico de que ficou grande demais pra 1 só.

2

Você fala 2 línguas operacionais distintas

Marketing + dev, comercial + ops, conteúdo + atendimento. Ele confunde os tons, mistura os contextos, perde precisão em ambas.

3

Você tem time/funcionários

Cada pessoa precisa de agente próprio · cada agente com escopo restrito ao papel da pessoa. Não dá pra todo mundo dividir o seu agente pessoal.

4

Você quer delegação 24/7

1 agente vigia inbox · outro escreve · outro atende comunidade. Cada um numa thread própria, sem se atropelarem.

APROFUNDAMENTO DO BLOCO 2

Subagente vs Agente Paralelo — *os 2 caminhos.*

A confusão clássica: aluno acha que "subagente" é "agente paralelo light". Não é. São **mecanismos distintos** com casos de uso diferentes. A tabela abaixo resolve a decisão.

ASPECTO	SUBAGENTE	AGENTE PARALELO
O que é		
Custo de criar		
Custo de manter		
Identidade	Limitada AGENTS.md TOOLS.md	
Memória	Sandbox isolado isolated fork	MEMORY.md
Aparece no Telegram?		
Como invocar	/subagents spawn <agentId> "<task>" sessions_spawn	
Aluno invoca direto?		
Ideal pra		

ASPECTO	SUBAGENTE	AGENTE PARALELO
Risco de overengineering		

💡 **Frase pra colar na geladeira:** subagente é *execução background* com identidade limitada. Agente paralelo é *entidade separada* com identidade completa. Subagente é **técnica de produtividade**. Agente paralelo é **decisão de organização**.

Como decidir

- ✓ **Tarefa pontual?** → Subagente (sempre que dúvida, comece por aqui)
- ✓ **Domínio contínuo + 4 sinais do Bloco 1?** → Agente paralelo (e antes, releia o custo de manutenção)
- ✓ **Não consegue decidir?** → Subagente. Sempre. Você reverte com 1 comando · agente paralelo te custa semanas pra desmontar

📘 **Doc oficial:** [tools/subagents](#) · [concepts/multi-agent](#). Releia se quiser fundamento técnico além do que A13 cobre.

APROFUNDAMENTO DO BLOCO 3

Subagente em ação — *pesquisador*.

Caso prático que aluno PME entende na hora: você quer Aurora escrever post sobre "tendências IA pra PME 2026". Pesquisa profunda exige ler 10-15 fontes, comparar, sintetizar. Se Aurora fizer direto, esgota contexto. **Solução: subagente "pesquisador" que roda em background.**


```
$ /subagents spawn researcher "tendências IA pra PME 2026 – busca 10-15 fontes brasileiras e inglesas, sintetiza em 200 palavras, foca em ferramentas práticas pra empreendedor não-técnico" -- thinking medium
```

▶ Resposta é enviada instantaneamente: `runId` (resposta do subagente)
Assim que o subagente termina o trabalho, ele retorna o `runId` para a Aurora.

O que acontece passo a passo

1

Bruno pede pra Aurora

"escreve post LinkedIn sobre tendências IA PME 2026". Aurora detecta que precisa de pesquisa profunda — não tenta fazer direto.

2

Aurora dispara `/subagents spawn`

Você não precisa colar nada. Aurora coordena automaticamente. Recebe `runId`, segue trabalhando estrutura/voz/gancho do post.

3

Subagente roda em sandbox isolado

Sessão limpa (recap A9 · modo `isolated`). Lê 10-15 fontes · cada subagente rastreia tokens próprios · sem poluir contexto da Aurora.

4

Announce posta síntese

Subagente termina · posta síntese de 200 palavras no canal de origem · Aurora integra no post final. Tela split — Aurora limpa em cima, subagente embaixo cheio de fontes lidas.



Princípio do Bloco 3: subagente faz o trabalho sujo em paralelo. Aurora mantém contexto limpo. *Async* — Aurora não fica esperando. É produtividade pura.



Ressalva técnica importante: subagente herda `AGENTS.md` + `TOOLS.md` mas **NÃO** o SOUL/IDENTITY da Aurora. Significa que ele segue regras gerais do workspace, mas *sem personalidade própria*. Não é problema pra task técnica (pesquisar é pesquisar). Diferente de criar um "pesquisador agente paralelo" com identidade própria — esse caminho custa 5x mais manutenção.

APROFUNDAMENTO DO BLOCO 4

Onde os agentes vivem + *4 caminhos de comunicação.*

Bloco mais denso da aula — resolve as 2 maiores dúvidas do aluno: **onde os agentes vivem?** e **como conversam entre si?**

Onde eles vivem (por plataforma)

PLATAFORMA	LIMITE	COMO FUNCIONA
Managed	1 OpenClaw por plano	dentro do mesmo workspace
VPS standalone	3 OpenClaws na mesma VPS	

Pra aluno do curso (Managed): única opção é multi-agente NO MESMO WORKSPACE. Estrutura de pastas:

```

~/openclaw/
├── .env                                ← compartilhado (cuidado escopo)
├── shared/                            ← assets compartilhados (logos, template)
├── agents/
│   ├── amora/
│   │   ├── SOUL.md                    ← identidade direta
│   │   ├── IDENTITY.md                ← Chief of Staff
│   │   ├── MEMORY.md                  ← memória própria isolada
│   │   └── workspace/
│   ├── openclawzinho/
│   │   ├── SOUL.md                    ← identidade atenciosa (DIFERENTE)
│   │   ├── IDENTITY.md                ← Suporte da comunidade
│   │   └── ...
│   └── pesquisador/                  ← subagente – sem SOUL próprio

```

Como conversam entre si – 4 caminhos REAIS

 **Importante:** NÃO existe DM interna direta entre agentes em runtime. A comunicação acontece via mecanismos abaixo. Não invente flag `--from --to` – ela não existe na doc oficial.

CAMINHO	MECANISMO REAL	QUANDO USAR
1. Subagente via spawn	<pre>/subagents spawn <agentId> "<task>" sessions_spawn</pre>	
2. Mention em canal compartilhado	<pre>@amora @openclawzinho ... requireMention: true</pre>	
3. Arquivo compartilhado	<pre>~/ .openclaw/shared/notes/X.md</pre>	
4. Trigger via cron + flag	<pre>openclaw agent --agent <id></pre>	

A pergunta-armadilha clássica do aluno

"Se eu colocar Amora E openclawzinho no mesmo grupo do Telegram, eles vão ver as mensagens um do outro?"

Resposta direta:

- ✓ Tecnicamente, **sim** — ambos os bots leem o canal
- ✓ Por design (recap A4 — `requireMention: true`), **só respondem quando mencionados**: `@amora` ou `@openclawzinho`
- ✓ Mensagem genérica sem mention? **Ninguém responde** (não vira spam de 2 respostas duplicadas)
- ✓ Pra cooperarem em 1 thread: `@amora @openclawzinho coordenem essa task` — ambos veem, mas só o mencionado primeiro responde por default
- ✓ `AGENTS.md` pode configurar handoff (ex: openclawzinho escala pra Amora se aparecer mention dela)

💡 **Princípio do Bloco 4:** por default, agentes não se veem em runtime. *Você decide* quando e como cooperam — via subagente, mention explícita, arquivo compartilhado ou cron. Previne spam, vazamento de contexto entre escopos (recap A10) e loops infinitos.

APROFUNDAMENTO DO BLOCO 5

AGENTS.md como *organograma vivo*.

Recap A5: você criou `AGENTS.md` no day 1 com framing "preparado pra equipe".
Hoje ganha substância — vira o organograma vivo do seu time digital.

O que tem em AGENTS.md

SEÇÃO	CONTEÚDO
Quem é cada agente	
Quem chama quem	
Onde cada um vive	
Permissões cruzadas	
Governance	

AGENTS.md real (sanitizado)

AGENTS.md – Pixel Educação

Amora COS (Chief of Staff)

- Modelo: Claude Sonnet 4.5 / GPT 5.5
- Localização: ~/.openclaw/agents/amora/
- Papel: coordenação geral, contexto Bruno, propagação semântica
- Subagentes invocáveis: pesquisador, gerador-imagem, analista
- Agentes paralelos coordenados: openclawzinho

openclawzinho

- Modelo: GPT-5.5
- Localização: ~/.openclaw/agents/openclawzinho/
- Papel: atendimento da comunidade do curso
- Canal: WhatsApp comunidade (recap A10 – monitoramento diário 30d)
- Handoff pra Amora: escala se questão fora de escopo

pesquisador (subagente)

- Modelo: Claude Haiku
- Tipo: subagente · invocado via /subagents spawn
- Modo: isolated · retorna síntese de 200 palavras



Princípio do Bloco 5: AGENTS.md é vivo. Cresce com o time. Source of truth de quem faz o quê. Quando você atualiza um agente, atualiza aqui também — senão Amora não sabe o que mudou.

APROFUNDAMENTO DO BLOCO 6

Caso real Pixel — *3 agentes em produção.*

Bruno narra o que tem hoje, sanitizado, em ranges. Não como vitrine — como **prova viva** de que dá pra ter time digital sem virar bagunça.

Os 3 agentes principais

🕒 COORDENAÇÃO

Amora COS

Chief of Staff pessoal. Coordena tudo · contexto Bruno · propagação semântica · orquestra os outros agentes. Subagentes invocáveis: pesquisador, gerador-imagem, analista.

🕒 ATENDIMENTO

openclawzinho

Suporte da comunidade do curso. Atende WhatsApp · escopo restrito a dúvidas de OpenClaw · escala pra Amora se sai do escopo. Monitorado diariamente nos primeiros 30d (recap A10).

🕒 OPERAÇÕES

Leonardo Da Vinci

Pixel Ops. Agente da equipe (compartilhado entre sócios) · acessa kanban · métricas SaaS · análise de dados · relatórios pro time. Slack interno como canal.

Como funcionam

- ✓ **Workspace separado por agente** (mesma `~/ .openclaw/`, pastas isoladas)
- ✓ **Identidades distintas** (Amora direta · openclawzinho atencioso · Leonardo executor)
- ✓ **Permissões cruzadas**: Amora vê tudo · openclawzinho só comunidade · Leonardo só ops
- ✓ **4 caminhos de comunicação**: subagente spawn + arquivo compartilhado em `shared/notes/` + mention em canais compartilhados + cron

Custo real (ranges sanitizados)

AGENTE	RANGE MENSAL	CAP DEFINIDO
Agente principal		<code>AGENTS.md</code>
Agente atendimento		
Agente especialista		

- ✓ **Princípio do Bloco 6**: 3 agentes ≠ 3x custo. Cada um roda quando precisa. Cap mensal por agente (governance no `AGENTS.md`). Total previsível, audit log diário, digest semanal mostrando consumo por agente. *Sem surpresa.*

A PROFUNDAMENTO DO BLOCO 7

6 sintomas de *overengineering*.

Se você tá em multi-agente e vê algum desses sinais, **volta pra 1**. Não tem vergonha — todo mundo passa por isso.

Recap em 4 linhas

- ✓ **Maioria dos PME: 1 agente.** Multi-agente é refinamento – custo real é manutenção contínua, não setup
- ✓ **2 caminhos:** subagente (background com identidade limitada) vs agente paralelo (entidade separada com identidade completa)
- ✓ **No Managed:** agentes paralelos vivem no mesmo workspace (1 OpenClaw, N agentes lá dentro). Comunicação via 4 caminhos (subagente spawn · mention · arquivo · cron) – NÃO há DM interna direta
- ✓ **AGENTS.md** é o organograma vivo – preparado desde A5, agora com substância



Frase de fechamento: 1 agente bem configurado primeiro. Subagente quando precisa especialista pontual. Agente paralelo só quando provar que vale a manutenção 5x.

6 prompts do *cheatsheet multi-agente*.

Resumo dos prompts canônicos do [multi-agente.md](#). Cola direto no agente — ele conduz a partir daí.



PROMPT #1 · ANÁLISE "PRECISO EVOLUIR?"

COLA NO AGENTE PRINCIPAL

Quero analisar honestamente se preciso evoluir pra multi-agente.

Contexto: [DESCREVE TUA OPERAÇÃO ATUAL EM 3 LINHAS – o que tu faz com 1 agente hoje, onde tá funcionando bem, onde tá travando].

Avalia os 4 sinais da A13:

1. Agente único tá lento (context rot · threads pesadas)?
2. Eu falo 2 línguas operacionais distintas (ex: marketing + dev)?
3. Tenho time/funcionários que precisam de agente próprio?
4. Quero delegação 24/7 (1 vigia inbox · outro escreve · outro atende)?

E o custo de manutenção 5x:

- Vou aceitar 3x SOULs pra sincronizar?
- 3x rotações de credenciais?
- 3x monitoramento diário primeiros 30d?
- 3x skills pra manter atualizadas?

Resposta honesta: vale a pena agora OU é melhor ficar com 1 e revisar em 90 dias? Sem hype. Sem arrogância. Só análise.



Se quiser, pode copiar e colar esse prompt no agente principal. Ele vai te ajudar a decidir se precisa evoluir para multi-agente.



PROMPT #2 · CRIAR SUBAGENTE ESPECIALISTA

COLA NO AGENTE PRINCIPAL

Quero criar um subagente especialista em [DOMÍNIO – ex: pesquisa profunda, análise PDF, geração de imagem].

Contexto: tarefa pontual, async, sandbox isolado. Subagente herda só AGENTS.md + TOOLS.md (sem SOUL próprio).

Me ajuda a:

1. Definir o agentId (handle curto · sem espaço · em inglês)
2. Escrever a descrição que vai ficar em AGENTS.md (3–5 linhas)
3. Listar as TOOLS específicas que ele precisa (não herda allowlist do agente pai)
4. Decidir o modo: isolated (sandbox limpo · default) ou fork (herda contexto pai · pra continuação de tarefa)
5. Mostrar o comando final /subagents spawn pra eu testar

Quero comando funcional, não pseudocódigo.



Quero que você crie um subagente especialista em [DOMÍNIO] que herda o contexto do agente principal e execute a tarefa de forma assíncrona e isolada. O subagente deve usar o mesmo SOUL que o agente principal e herdar os arquivos AGENTS.md e TOOLS.md. O comando final deve ser /subagents spawn pra eu testar.



Quero gerar um AGENTS.md completo pro meu workspace – organograma vivo do meu time digital.

Contexto: tenho [N] agentes hoje: [LISTA – handle, papel, modelo, canal].

Mais [M] subagentes invocáveis: [LISTA – handle, tipo de tarefa].

Cria AGENTS.md com as 5 seções da A13:

1. Quem é cada agente (handle · papel · modelo · escopo)
2. Quem chama quem (subagentes disponíveis · agentes paralelos com handoff)
3. Onde cada um vive (workspace · canais · integrações)
4. Permissões cruzadas (scope por agente – recap A10)
5. Governance (cap mensal · audit log · digest semanal)

Formato Markdown, pronto pra colar em ~/.openclaw/AGENTS.md.



Quero gerar um AGENTS.md completo pro meu workspace – organograma vivo do meu time digital.



Quero configurar handoff entre 2 agentes paralelos: [agente A – handle]
e [agente B – handle].

Cenário: agente A é [PAPEL – ex: atendimento da comunidade] · escopo restrito a [DOMÍNIO]. Quando aparecer questão fora desse escopo, ele deve escalar pra agente B (que é [PAPEL – ex: coordenação geral]).

Me ajuda a:

1. Definir os triggers de escala (palavras-chave · padrões de pergunta · menção explícita do nome de B)
2. Escrever a regra em AGENTS.md (seção handoff de A pra B)
3. Mostrar como fica no SOUL.md de A (como ele "sabe" quando escalar)
4. Mostrar exemplo prático: aluno pergunta X em canal · A reconhece · A responde com handoff (mention @B · contexto resumido)
5. Configurar requireMention: true no canal compartilhado pra evitar spam de respostas duplicadas

Sem inventar flag --from --to (não existe). Só usar mecanismos reais da doc OpenClaw.



Exemplo de configuração para o canal compartilhado (SOUL.md de A):

```
requireMention: true
```



Quero criar um SEGUNDO agente paralelo dedicado APENAS ao canal Telegram
[DESCREVE O CANAL: ex "grupo da comunidade do meu curso", "grupo do time de atendimento", "DM do meu cliente X"].

NÃO quero que ele acesse outros canais da [Amora / agente principal].

NÃO quero que ele veja contexto pessoal meu (USER.md fica isolado).

Workspace, identidade e memória SEPARADOS.

Vai na doc oficial OpenClaw + cheatsheet multi-agente.md (Prompt #5 completo) e executa em ordem:

1. Bot novo no @BotFather (NÃO reusa token da Amora) · me lembra de
rodar /setprivacy → Disable ANTES de adicionar no grupo (recap A1/A4)
2. Workspace isolado em ~/.openclaw/agents// com SOUL/IDENTITY/MEMORY próprios. SOUL com TOM adequado ao papel (suporte = paciente · equipe = direto operacional · cliente = profissional formal)
3. Conecta SÓ a este canal · escopo restrito ao chat_id · privacy disabled · requireMention: true se for grupo
4. Registra em AGENTS.md com nome · papel · canais · escopo · cap mensal de custo · regras de handoff com Amora
5. Validação: manda mensagem no canal (novo agente responde) · manda
em outro canal da Amora (novo agente NÃO responde lá)

Princípios A10: scope mínimo · identidade própria isolada · cap mensal
(default R\$ 50/mês – pergunta o valor) · audit log primeiros 30 dias.

NÃO crie sem mostrar o plano completo antes. Quero confirmar identidade,
escopo e cap antes de você executar.



Minha [Amora / agente principal] tá saturada – começou a misturar contextos do canal [DESCREVE O CANAL] com o resto. Sintomas: [ex: "responde no tom da minha persona pessoal pra cliente", "esquece briefing do grupo X depois de 2 dias", "não distingue thread do cliente Y do thread do cliente Z"].

Quero migrar TODO o contexto desse canal pra um agente novo dedicado.

Vai na doc oficial + cheatsheet multi-agente.md (Prompt #6 completo) e

executa em ordem (CONFIRMA cada passo antes de seguir):

1. Análise prévia (sem mexer em nada) · lista o que migra vs fica · tempo + risco
2. Cria o agente novo seguindo o Prompt #5 (bot novo, privacy disabled, workspace isolado, identidade própria) · usa nome [SUGESTÃO]
3. Migra arquivos · MOVE (não copia) · backup em shared/migrations/
/ antes de mover · atualiza referências internas
4. SOUL adequado ao tom do canal · MEMORY já populado com histórico relevante (últimos 30 dias)
5. Desconecta a Amora desse canal · atualiza AGENTS.md · regra de handoff (Amora encaminha pro novo agente se cliente DM ela acidentalmente)
6. Validação: manda 1 mensagem teste · novo agente responde · Amora ainda funciona normal nos outros canais dela

CRÍTICO: NÃO faz migração destrutiva sem backup. NÃO desconecta a Amora

do canal antes do novo agente estar 100% testado e respondendo.

Aluno

PME não pode ficar com canal mudo durante migração.





Cheatsheet completo: multi-agente.md – análise honesta "preciso evoluir?" + 4 sinais + 6 sintomas overengineering · tabela prós/contras subagente vs paralelo · arquitetura por plataforma · 4 caminhos REAIS de comunicação · template AGENTS.md de organograma · troubleshooting comum · 6 prompts prontos pra colar.

TRAVOU? ANTES DE PEDIR AJUDA

Top *problemas comuns da A13.*

SINTOMA	CAUSA PROVÁVEL	AÇÃO
<code>/subagents</code> <code>spawn</code>	<code>AGENTS.md</code>	<code>AGENTS.md</code>
	<code>requireMention:</code> <code>true</code>	
	<code>AGENTS.md</code>	
		<code>requireMention</code>
	<code>AGENTS.md</code>	



Não conseguiu resolver com a tabela acima? Joga screenshot/ áudio/vídeo no [grupo Telegram do curso](#). openclawzinho responde em < 30 segundos · humanos em < 5 minutos.

Próximo passo: *A14 — Mission Control*

15-20 minutos · modular · aula reaproveitada do V1. Você tem 1 OU vários agentes operando 24/7 — como você OBSERVA tudo? Sem ficar olhando 5 telas de Telegram, sem perder o que cron rodou ontem, sem adivinhar se erro ficou silencioso. **Mission Control** é o cockpit conversacional onde você constrói dashboard customizado com a Amora — workflow completo via áudio Telegram.

→ Ir pra Aula 14

TRAVOU? TÁ NO GRUPO DO CURSO

Comunidade Telegram do curso — t.me/courseopenclaw — suporte 24/7 (openclawzinho bot + Bruno e equipe presente). Dúvida sobre essa aula, problemas de setup, ou quer compartilhar como aplicou no seu negócio: bora trocar lá.

PRÓXIMO NÍVEL: PIXEL AI HUB

Aplicar agente pra empresa toda — equipe, governance, escala? [Pixel AI Hub](#): segundo cérebro corporativo, comunidade de empreendedores, +50 skills enterprise, encontros mensais ao vivo. Vagas mensais limitadas.